

Project 07: RAG Pipeline + MCP Server — Full Build Walkthrough

This is a full chronological record of building the RAG pipeline with MCP functionality on Cloud Run, including every command that worked, why it worked, what failed along the way, and why.

1. Conceptual Foundation

RAG pipeline stages (5 broad components):

Data ingestion

RAG pipeline (Retrieval, Augmentation, Generation)

Postprocessing (safety filters, custom checks, RAGAS — deferred)

GCP observability (deferred, not Terraform-provisioned)

R/A/G breakdown (inputs/outputs/tools):

Retrieval: tool = pgvector similarity search via LangChain retriever → input = user query → output = top-k relevant chunks

Augmentation: tool = LCEL prompt template → input = query + top-k chunks → output = filled prompt

Generation: tool = ChatVertexAI (Gemini) → input = filled prompt → output = answer text

MCP concept: Wraps the entire R→A→G→postprocessing pipeline as a single callable "tool" behind an MCP server. Any MCP-compatible model (Claude, GPT, Gemini) can call it as an endpoint — analogous to a human calling a REST API. The internal generation step still uses Gemini via ChatVertexAI; MCP doesn't remove that, it just adds an external calling layer on top. No duplicated generation work occurs — the calling model delegates the "answer using my private data" task entirely; it never generates and discards its own answer.

MCP Python SDK: created and maintained by Anthropic (open-sourced under `modelcontextprotocol` GitHub org). Installed v2 (`pip install "mcp[cli]"`), which uses `from mcp.server import MCPServer` (not v1's `FastMCP`).

2. Architecture Decisions (locked in early)

Cloud SQL: private IP only (`ipv4_enabled = false`), no public IP at all — reachable only via VPC.

IAM database authentication (`cloudsql.iam_authentication` flag = "on") instead of password auth — ties DB access to service account identity.

`secrets.tf` **deleted entirely** — no password to store once IAM auth was adopted.

Private Service Access: reserved IP range (`google_compute_global_address`) + `google_service_networking_connection` peering VPC to Google's service producer network — required for Cloud SQL private IP to attach to a network at all.

Serverless VPC Access connector: lets Cloud Run (which lives outside the VPC by default) reach the private Cloud SQL IP.

Cloud Run: `--allow-unauthenticated` (IAM `roles/run.invoker` granted to `allUsers`) since external models can't present GCP identity tokens. Auth enforced at the **application layer** instead (planned API key/OAuth check inside `query_rag_pipeline`, still deferred/TODO).

CI+CD combined in one `cloudbuild.yaml`: build → push → `gcloud run deploy`, all in one file. This is a deliberate departure from the pattern in other portfolio projects (04/05, 06) where Cloud Build only does CI (build/push, then triggers a Vertex AI job) — here Cloud Run is a persistent service, so Cloud Build does true CD too. Cloud Deploy (managed multi-env progressive rollout) was considered but rejected as

overkill for a single-environment POC — flagged as a good fit for iter3's dev/staging/prod work instead.

Ingestion/embedding kept separate from the MCP tool: `ingest.py`/`chunk.py`/`embed.py` run as an occasional/one-off Cloud Run **Job**, not as part of the live `query_rag_pipeline` MCP tool call, to avoid re-scanning/re-embedding documents on every query and to keep the query path fast and stateless.

3. Terraform Files (final state, by file)

`main.tf`

Backend: GCS bucket `rag-pipe-mcp-terraform-state`, prefix `rag-mcp`. Provider `google` `~> 5.0`, `random` `~> 3.6`. Bucket pre-created manually before `terraform init`.

`variables.tf`

`project_id`, `region` (`eu-west-4`), `zone`, `ingestion_bucket_name`, `db_tier` (`db-f1-micro`), `db_name` (`ragdb`), `project_number`. `name_prefix` and `db_user` variables were both removed as unused once IAM auth replaced password auth and per-resource naming used `var.project_id` directly instead.

`apis.tf`

Enabled: `aiplatform`, `sqladmin`, `storage`, `documentai`, `logging`, `monitoring`, `iam`, `servicenetworking`, `run`, `vpcaccess`, `artifactregistry`, `cloudbuild`, **compute** (added late — see bug section), and **cloudresourcemanager** (enabled manually via `gcloud`, not Terraform, since it's the API that lets Terraform manage other APIs' enablement — chicken-and-egg problem).

`network.tf`

```
hcl

resource "google_compute_network" "myvpc" {
  name          = "${var.project_id}-vpc"
  auto_create_subnetworks = false
}

resource "google_compute_global_address" "private_ip_range" {
  name          = "${var.project_id}-private-ip-range"
  purpose       = "VPC_PEERING"
  address_type  = "INTERNAL"
  prefix_length = 16
  network       = google_compute_network.myvpc.id
}

resource "google_service_networking_connection" "private_vpc_connection" {
  network               = google_compute_network.myvpc.id
  service               = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.private_ip_range.name]
  depends_on            = [google_project_service.servicenetworking]
}
```

`database.tf`

```
hcl
```

```

resource "google_sql_database_instance" "rag_pg" {
  project      = var.project_id
  name         = "${var.project_id}-pg"
  region       = var.region
  database_version = "POSTGRES_15"
  deletion_protection = false

  settings {
    tier = var.db_tier
    ip_configuration {
      ipv4_enabled = false
      private_network = google_compute_network.myvpc.id
    }
    database_flags {
      name = "cloudsql.iam_authentication"
      value = "on"
    }
  }
}

depends_on = [google_project_service.sqladmin, google_service_networking_connection.private_vpc_co
}

```

```

resource "google_sql_database" "rag_db" {
  project = var.project_id
  name    = var.db_name
  instance = google_sql_database_instance.rag_pg.name
}

```

```

resource "google_sql_user" "rag_user" {
  project = var.project_id
  name    = replace(google_service_account.pipeline_sa.email, ".gserviceaccount.com", "")
  instance = google_sql_database_instance.rag_pg.name
  type    = "CLOUD_IAM_SERVICE_ACCOUNT"
}

```

Note: GCP requires the IAM DB username to have `.gserviceaccount.com` stripped, even at the Terraform `google_sql_user` resource level (not just in Python client code) — confirmed via error: "Database username for Cloud IAM service account should be created without '.gserviceaccount.com' suffix".

iam.tf

Service account `rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com` with roles: `storage.objectAdmin`, `aiplatform.user`, `cloudsql.client`, `cloudsql.instanceUser`, `logging.logWriter`, `monitoring.metricWriter`, `documentai.apiUser`. Plus `google_cloud_run_v2_service_iam_member.public_invoker` granting `roles/run.invoker` to `allUsers`.

storage.tf

Ingestion bucket (`rag-pipe-mcp-ingestion`, versioning enabled, uniform bucket-level access) + IAM binding granting `pipeline_sa` `storage.objectAdmin` on it.

artefactregistry.tf

```
hcl
resource "google_artifact_registry_repository" "mcp_server_repo" {
  project      = var.project_id
  location     = var.region
  repository_id = "mcp-server-repo"
  format       = "DOCKER"
  depends_on = [google_project_service.artifactregistry]
}
```

cloudbuild.tf

VPC Access connector ([10.8.0.0/28]) + [google_cloud_run_v2_service.mcp_server] running the MCP server image, with [service_account = pipeline_sa], [vpc_access] block (egress [PRIVATE_RANGES_ONLY]), and env vars: [PROJECT_ID], [REGION], [CLOUD_SQL_CONNECTION_NAME], [DB_NAME], [DB_USER] (full SA email — stripped in Python), [INGESTION_BUCKET_NAME].

cloudbuild_job.tf (added later, for ingestion)

```
hcl
resource "google_cloud_run_v2_job" "embed_job" {
  project = var.project_id
  name    = "${var.project_id}-embed-job"
  location = var.region
  template {
    template {
      service_account = google_service_account.pipeline_sa.email
      vpc_access {
        connector = google_vpc_access_connector.connector.id
        egress    = "PRIVATE_RANGES_ONLY"
      }
    }
    containers {
      image = "europe-west4-docker.pkg.dev/${var.project_id}/mcp-server-repo/mcp-server:latest"
      command = ["python"]
      args    = ["embed.py"]
      env { name = "PROJECT_ID" value = var.project_id }
      env { name = "REGION" value = var.region }
      env { name = "CLOUD_SQL_CONNECTION_NAME" value = google_sql_database_instance.rag_pg.conn }
      env { name = "DB_NAME" value = var.db_name }
      env { name = "DB_USER" value = google_service_account.pipeline_sa.email }
      env { name = "INGESTION_BUCKET_NAME" value = google_storage_bucket.ingestion.name }
    }
  }
}
```

Reuses the same Docker image as the MCP server — only the command/args differ. This is the key insight:

one image, multiple entrypoints (MCP service vs. one-off batch job).

outputs.tf

sql_instance_connection_name, pipeline_sa_email, ingestion_bucket_name, mcp_server_url

(added later for the two-pass deploy).

4. Python Pipeline Files (final state)

ingest.py

Downloads PDFs from GCS bucket, loads via PyPDFLoader, returns list of Document objects with source metadata pointing to gs:// path. Reads PROJECT_ID and INGESTION_BUCKET_NAME from os.environ.

chunk.py

Pure Python, no GCP calls. RecursiveCharacterTextSplitter, chunk_size=1000, overlap=150.

embed.py

Uses google.cloud.sql.connector.Connector + SQLAlchemy engine with enable_iam_auth=True and ip_type="PRIVATE" (critical — see bug section). Reads PROJECT_ID, REGION, CLOUD_SQL_CONNECTION_NAME, DB_NAME, DB_USER from env; strips .gserviceaccount.com from DB_USER in Python. Embeds via VertexAIEmbeddings (text-embedding-004), stores via langchain_postgres.PGVector (deferred decision: kept this generic library + manual connector wiring rather than switching to Google-native langchain-google-cloud-sql-pg).

generate.py

Retrieval (vectorstore.as_retriever) → augmentation (ChatPromptTemplate) → generation (ChatVertexAI, gemini-2.5-flash) via LCEL chain. Logs to Cloud Logging. Has a heuristic indicates_insufficient_context() check (lightweight string matching, not a real classifier).

mcp_wrapper.py

EO
56

python

```

import os
import uvicorn
from mcp.server import MCPServer
from mcp.server.transport_security import TransportSecuritySettings
from generate import ask

mcp = MCPServer("rag-pipeline-mcp")

@mcp.tool()
def query_rag_pipeline(question: str) -> dict:
    result = ask(question)
    # TODO: postprocessing hook (safety filter, custom checks)
    # TODO: API key / OAuth check
    return result

if __name__ == "__main__":
    port = int(os.getenv("PORT", 8080))
    security = TransportSecuritySettings(
        allowed_hosts=[
            "rag-pipe-mcp-mcp-server-hkigjsojqa-ez.a.run.app",
            "rag-pipe-mcp-mcp-server-hkigjsojqa-ez.a.run.app:*",
            "rag-pipe-mcp-mcp-server-514728048358.europe-west4.run.app",
            "rag-pipe-mcp-mcp-server-514728048358.europe-west4.run.app:*",
        ],
        enable_dns_rebinding_protection=True,
    )
    app = mcp.streamable_http_app(transport_security=security)
    uvicorn.run(app, host="0.0.0.0", port=port)

```

Critical bug fix: originally passed `transport_security=security` as a kwarg to `mcp.run(transport="streamable-http", ...)`. This silently did NOT apply the security settings — the fix was building the ASGI app explicitly via `mcp.streamable_http_app(transport_security=security)` and running it directly with `uvicorn.run()`, matching the documented pattern exactly.

`create_extension.py` (one-off admin script, added late)

Connects as `postgres` superuser (password auth, `ip_type="PRIVATE"`) to run `CREATE EXTENSION IF NOT EXISTS vector;` and `GRANT ALL ON SCHEMA public TO "<pipeline_sa_stripped_email>";` — both operations `pipeline_sa` (IAM user) lacks privilege to do itself.

`requirements.txt`

google-cloud-storage, google-cloud-logging, langchain-core, langchain-community, langchain-text-splitters, langchain-google-vertexai, langchain-postgres, cloud-sql-python-connector[pg8000], sqlalchemy, pg8000, pypdf, mcp[cli], uvicorn.

Dockerfile



dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY components/ .
ENV PORT=8080
EXPOSE 8080
CMD ["python", "mcp_wrapper.py"]
```

`COPY components/ .` flattens contents of `components/` directly into `/app` (not nested) — matches all scripts' flat same-directory imports (`from embed import ...`).

`cloudbuild.yaml`

```
EO
SE

yaml
steps:
- name: 'gcr.io/cloud-builders/docker'
  args: ['build', '-t', 'europe-west4-docker.pkg.dev/$PROJECT_ID/mcp-server-repo/mcp-server:latest', './pro
- name: 'gcr.io/cloud-builders/docker'
  args: ['push', 'europe-west4-docker.pkg.dev/$PROJECT_ID/mcp-server-repo/mcp-server:latest']
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  entrypoint: gcloud
  args: ['run', 'deploy', 'rag-pipe-mcp-mcp-server', '--image=europe-west4-docker.pkg.dev/$PROJECT_ID/m
images:
- 'europe-west4-docker.pkg.dev/$PROJECT_ID/mcp-server-repo/mcp-server:latest'
options:
  logging: CLOUD_LOGGING_ONLY
```

Build context path had to be the full nested repo path (`./projects/RAG_pipeline/pipeline`) since the Cloud Build trigger runs from repo root, and the Dockerfile/components live nested under `projects/RAG_pipeline/pipeline/`.

5. Bugs Hit and Fixes — Chronological

Terraform / Infra Setup

`terraform plan -target=...` **"Invalid target" error** — PowerShell was mangling the flag. Fix: quote it (`-target="resource.name"`) or use a space instead of `=`.

Cloud Build API not enabled — first `gcloud builds submit` prompted to enable `cloudbuild.googleapis.com`; accepted (`y`).

NOT_FOUND: Spanner: ServiceAccountInfoDataType — transient error right after enabling Cloud Build API; the default Cloud Build service account hadn't finished provisioning. Fix: wait ~1-2 min, retry.

`storage.objects.get` **denied on _cloudbuild bucket** — Compute Engine default SA (`514728048358-compute@developer.gserviceaccount.com`) lacked read access to its own build staging bucket. Fix:

```
EO
SE

gcloud projects add-iam-policy-binding rag-pipe-mcp --member="serviceAccount:514728048358-compute@developer.gserviceaccount.com" --role="roles/storage.objectViewer"
```

COPY failed: requirements.txt not found — simple typo in filename in the pipeline directory. Fixed by correcting the typo, rerunning the same build command.

artifactregistry.repositories.uploadArtifacts denied — Compute default SA lacked Artifact Registry write access. Fix:

```
EO
56
gcloud projects add-iam-policy-binding rag-pipe-mcp --member="serviceAccount:514728048358-compute@developer.gserviceaccount.com" --role="roles/artifactregistry.writer"
```

Compute Engine API has not been used... SERVICE_DISABLED during `terraform apply` on `google_compute_network` — `compute.googleapis.com` was never added to `apis.tf`. Fix: added the resource, applied.

Failed to upload state to gs://...: dial tcp: lookup storage.googleapis.com: no such host

mid-apply — local DNS/network dropped during a long-running apply (SQL instance creation, ~10+ min). Terraform wrote `errored.tfstate` locally as a safety backup. Recovery:

Confirmed network was back (`ping`, `nslookup`).

`terraform state push errored.tfstate` — failed initially with a stuck lock error (`Error acquiring the state lock... conditionNotMet`).

`terraform force-unlock <LOCK_ID>` — resolved the lock.

Confirmed the SQL instance had actually finished creating on GCP's side via `gcloud sql instances list` (status: RUNNABLE) despite the local apply failure.

`terraform plan` then showed it wanting to *recreate* the SQL instance (since state was stale) — fixed via:

```
EO
56
terraform import google_sql_database_instance.rag_pg rag-pipe-mcp/rag-pipe-mcp-pg
```

After import, `terraform plan` showed only a harmless in-place update (`deletion_protection: true -> false`) plus the genuinely new resources (db, user, Cloud Run service, IAM binding) — applied successfully.

Error, failed to insert user ...gserviceaccount.com into instance: Database username for

Cloud IAM service account should be created without ".gserviceaccount.com" suffix — `google_sql_user.rag_user`'s `name` needed the suffix stripped even in Terraform, not just Python:

```
EO
56
hcl
name = replace(google_service_account.pipeline_sa.email, ".gserviceaccount.com", "")
```

Cloud Run deploy PERMISSION_DENIED: run.services.get during first automated Cloud Build (CI+CD)

run — the build's *actual* identity was `pipeline_sa` (not the default Compute SA), since the Cloud Build trigger was configured to use `pipeline_sa` as its build service account. Fixed by granting `pipeline_sa`:

```
EO
56
gcloud projects add-iam-policy-binding rag-pipe-mcp --member="serviceAccount:rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com" --role="roles/run.admin"
gcloud iam service-accounts add-iam-policy-binding rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com --member="serviceAccount:rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com" --role="roles/iam.serviceAccountUser"
```

artifactregistry.repositories.downloadArtifacts denied on Cloud Run deploy (`pipeline_sa` pulling the image) — fixed:

```
EO
56
```

```
gcloud projects add-iam-policy-binding rag-pipe-mcp --member="serviceAccount:rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com" --role="roles/artifactregistry.reader"
```

artifactregistry.repositories.uploadArtifacts **denied** on the final **docker push** step of the same automated build (after deploy succeeded) — **pipeline_sa** also needed writer, not just reader:

```
gcloud projects add-iam-policy-binding rag-pipe-mcp --member="serviceAccount:rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com" --role="roles/artifactregistry.writer"
```

Cloud Build trigger: **unable to prepare context: unable to evaluate symlinks in Dockerfile path** — build context in **cloudbuild.yaml** was **.** (repo root), but Dockerfile actually lived nested at **projects/RAG pipeline/pipeline/**. Diagnosed via a debug step (**find /workspace -maxdepth 2**) added temporarily to **cloudbuild.yaml**. Fixed by pointing build context at the correct nested path (with quoting/escaping for the space in the folder name).

could not resolve source... storage.objects.get (**again, different bucket/account**) and other transient service errors during the **second wave** of **terraform apply**s (Cloud Shell session, next day) — a mix of genuine propagation delay (**cloudresourcemanager.googleapis.com** had never been enabled at all — required manual **gcloud services enable cloudresourcemanager.googleapis.com** since it's the API that lets Terraform manage other APIs' state) and pure transient/flaky errors (different resources failing on each retry, IPv6 DNS blips: **cannot assign requested address**). Resolved purely by enabling the API and retrying **terraform apply** repeatedly until it succeeded — no config changes needed for the flaky ones.

Cloud Shell: **terraform init** kept showing "install terraform" prompt even after installing — **PATH** had 3 **terraform** binaries (**/usr/bin**, **/bin**, **/google/bin**), and the shell had cached (hashed) an old lookup from before install. Fixed with **hash -r** to clear the shell's command cache.

Application-Layer / Runtime Bugs

ModuleNotFoundError: No module named 'pypdf' locally — **pypdf** wasn't in **requirements.txt** at first (it's a transitive need of **PyPDFLoader**, not directly imported). Fixed: **pip install pypdf**.

Local embed.py run: **Unable to acquire impersonated credentials...**

iam.serviceAccounts.getAccessToken denied — attempted to impersonate **pipeline_sa** locally (since **embed.py** connects using IAM auth as that SA) without first granting the local Gmail account **roles/iam.serviceAccountTokenCreator** on **pipeline_sa**. Fixed:

```
gcloud iam service-accounts add-iam-policy-binding rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com --member="user:yahya.sekinat@gmail.com" --role="roles/iam.serviceAccountTokenCreator"

gcloud auth application-default login --impersonate-service-account=rag-pipe-mcp-pipeline@rag-pipe-mcp.iam.gserviceaccount.com
```

Key concept clarified: impersonation always needs a real authenticated identity behind it first (you log in as yourself, then Google mints tokens *as* the target SA on your behalf) — you can't "log in as" a service account directly since SAs have no interactive login of their own.

18. **CloudSQLITypeError: Cloud SQL instance does not have any IP addresses matching preference: PRIMARY** — the Cloud SQL Python Connector defaults to preferring a PUBLIC IP unless told otherwise; this instance has private IP only. Fixed by adding **ip_type="PRIVATE"** to the

`connector.connect(...)` call in `embed.py` (and later `create_extension.py`).

TimeoutError: [Errno 110] Connection timed out running `embed.py` from **Cloud Shell** even after the `ip_type="PRIVATE"` fix — Cloud Shell is NOT inside the VPC and cannot reach the private IP, same fundamental issue as the local laptop. This led to the decision to run ingestion via a **Cloud Run Job** instead (reusing the existing VPC-connected Cloud Run infrastructure), since that's the only environment available that can actually reach the private Cloud SQL IP. Considered and rejected alternatives: VPC-native SSH tunnel via a Compute Engine VM (more manual infra to stand up) and Private Google Access setup.

421 Invalid Host header / Misdirected Request calling the deployed MCP server via `curl` and via the MCP Inspector — root cause was the `mcp_wrapper.py` bug in item (see `mcp_wrapper.py` section above: `transport_security` passed to `mcp.run()` instead of `mcp.streamable_http_app()`). Diagnosed by comparing the exact `Host` header sent (`curl -v ... | grep "^> Host"`) against the `allowed_hosts` list — they matched exactly, which was the clue that the setting wasn't actually being applied at all, not that the list was wrong.

Even after fixing `mcp_wrapper.py` and pushing, the 421 persisted — root cause: `gcloud run deploy` (from the CI/CD pipeline) had NOT picked up the newest pushed image, despite it being tagged `:latest` — a race condition between the `docker push` re-tagging `:latest` and the deploy step resolving the image reference. Diagnosed by comparing the digest Cloud Run was actually running (`gcloud run revisions describe <rev> --format="value(spec.containers[0].image)"`) against the newest digest in Artifact Registry (`gcloud artifacts docker images list ... --include-tags`) — they didn't match. Fixed by an explicit manual redeploy:

E056

```
gcloud run deploy rag-pipe-mcp-mcp-server --image=europe-west4-docker.pkg.dev/rag-pipe-mcp/mcp-server-repo/mcp-server:latest --region=europe-west4
```

E056

After this, digest comparison confirmed the correct image was running, and the `'curl'` test returned `'200 OK'`.

22. MCP Inspector UI showed "Forbidden" even after the Host header fix was confirmed working via

`curl` — Cloud Run logs showed NO entry at all for the Inspector's attempt (unlike the `curl` attempts, which did log). Concluded the rejection was happening client-side (browser/CORS or Cloud Shell's Web Preview proxy), not server-side. Rather than debug the Inspector/browser layer further, pivoted to a plain Python `mcp.Client` script instead — which worked immediately, confirming the server itself was fine.

permission denied to create extension "vector" ... Must be superuser — `pipeline_sa` (IAM DB user) cannot create Postgres extensions; only superuser (`postgres`) can. This required: resetting the `postgres` user's password (`gcloud sql users set-password postgres --instance=rag-pipe-mcp-pg --password=...`), then running `CREATE EXTENSION IF NOT EXISTS vector;` as that superuser.

Direct `psql`-via-Cloud-SQL-Proxy connection from Cloud Shell was attempted first but failed for the same private-IP-unreachable reason as item 19 (`gcloud sql connect` defaults to public IP; even with `--private-ip` flag, Cloud Shell still can't route to the VPC's private IP — `dial tcp 10.200.0.3:3307: i/o timeout`). Resolved the same way as ingestion: wrote a one-off `create_extension.py` script and ran it via the **same Cloud Run Job**, overriding its args:

E056

```
gcloud run jobs execute rag-pipe-mcp-embed-job --region=europe-west4 --args=create_extension.py --update-env-vars=POSTGRES_PASSWORD=<password>
```

```
EO
SE
```

Note: `gcloud run jobs execute` does NOT support a `--command` override flag (only `--args` and `--update-env-vars`) — first attempt with `--command=python` failed with "unrecognized arguments."

24. **permission denied for schema public** — after the vector extension was created, `embed.py`'s `PGVector` still failed trying to `CREATE TABLE` in the `public` schema, since `pipeline_sa` had no schema-level privileges. Fixed by extending `create_extension.py` to also run, as the `postgres` superuser:

```
EO
SE
```

```
sql
```

```
GRANT ALL ON SCHEMA public TO "rag-pipe-mcp-pipeline@rag-pipe-mcp.iam";
```

```
EO
SE
```

Re-ran the override execution, then re-ran the normal `embed.py` job execution — both succeeded.

25. **Minor: file not found errors in Cloud Run Job executions** (`create_extension.py` not found in `/app`)

— simply because the new file hadn't been committed/pushed to GitHub yet before triggering the job, so the image being pulled didn't contain it. Also hit a typo in the filename (missing/extra "s") on one pass.

Fixed by committing, pushing, waiting for the Cloud Build trigger to finish, then re-executing the job.

Started server process, stray `psql` connection attempts failing with `server closed the connection unexpectedly` — multiple root causes stacked across attempts: (a) stale Cloud SQL Proxy process left listening on the port from a previous session with an expired/stale token, fixed by killing it and restarting fresh; (b) the proxy defaulting to public IP (see item 18's proxy-CLI equivalent), fixed with `--private-ip` flag; (c) even with that fix, Cloud Shell fundamentally cannot route to the private IP at all (confirmed via the `dial tcp 10.200.0.3:3307: i/o timeout` error) — this is what forced the pivot to the Cloud Run Job approach for the extension/grant setup rather than continuing to fight direct `psql` access from Cloud Shell.

6. Final Working End-to-End Test

```
EO
SE
```

```
python
```

```

import asyncio
from mcp import Client

SERVER_URL = "https://rag-pipe-mcp-mcp-server-514728048358.europe-west4.run.app/mcp"

async def main() -> None:
    async with Client(SERVER_URL) as client:
        result = await client.call_tool(
            "query_rag_pipeline",
            {"question": "What roles is this candidate applying for?"},
        )
        print(result)

if __name__ == "__main__":
    asyncio.run(main())

```

Result: `is_error=False`, correct answer synthesized from 4 relevant cover letters, with accurate `gs://` source attribution — confirming the full pipeline (ingestion → chunking → embedding → retrieval → augmentation → generation → MCP tool wrapping → Cloud Run deployment → external client call) works end-to-end.

7. CI/CD Automation Setup (final)

Cloud Build trigger `RAG-MCP-trigger` connected to GitHub repo `Sekinat95/cloud-architect-portfolio-iter2`, watching branch `rag-pipe`, build config at `projects/RAG_pipeline/pipeline/cloudbuild.yaml`.

Every push to `rag-pipe` triggers: docker build (from the correct nested context) → push to Artifact Registry → `gcloud run deploy` to the MCP server service.

This is full CI **and** CD combined in one pipeline — a deliberate, explained departure from other portfolio projects' CI-only pattern (since those deploy to Vertex AI batch jobs, not a persistent service).

The **embed/ingestion Cloud Run Job** and the **create_extension.py one-off admin script** both reuse this exact same built image — no separate build pipeline — invoked via `gcloud run jobs execute` with different `--args` overrides as needed.

8. Key Reusable Learnings

IAM DB auth vs GCP IAM roles are two separate systems: GCP IAM role = "can this identity do X on GCP" (e.g. `cloudsql.client`). DB-level IAM auth = "can this identity log into this specific Postgres instance" (needs `cloudsql.instanceUser` role + DB flag + `google_sql_user` registration). Neither alone is sufficient.

Superuser-only Postgres operations (`CREATE EXTENSION`, schema `GRANT`s) cannot be done by an IAM-authenticated service account — they require the `postgres` superuser, reached via password auth.

Private-IP-only Cloud SQL is unreachable from anywhere outside the VPC — not just your laptop, but also Cloud Shell, by design. The only ways in are: something already inside the VPC (Cloud Run + VPC connector, a Compute Engine VM, GKE), or a VPN/Interconnect bridging an external network into the VPC.

Cloud Run Jobs are the right tool for one-off admin/batch tasks that need VPC access, reusing the exact same container image as the persistent service, just with a different command/args override at

execution time — no separate build or infra needed.

`:latest` **tag reuse across builds isn't atomic** — a deploy step can pull a stale `:latest` if it resolves the tag before a concurrent/recent push finishes re-tagging it. When in doubt after a rebuild, explicitly verify the running revision's actual image digest against the newest pushed digest before assuming a fix took effect.

Shell command caching (`hash -r`) and multiple binaries on PATH can cause a freshly-installed CLI tool to appear "not installed" — always check `type -a <tool>` and clear the hash table if the installed binary and the invoked one seem to disagree.